

Laporan Manipulasi Citra Digital

Pengolahan Sinyal Digital

Nama : Fatih Jawwad Al Mumtaz
NIM : 452024611047
Kelas : Teknik Informatika – Semester 5
Dosen Pengampu : Assoc. Prof. Dr. Oddy Virgantara Putra, S.Kom., M.T.

1 Pendahuluan

Manipulasi citra digital adalah proses ngubah nilai pixel pada gambar pake operasi matematika. Operasinya bisa linier, non-linier, atau aritmetika sederhana tergantung efek yang mau dicapai. Laporan ini dokumentasi eksperimen empat teknik manipulasi citra pake OpenCV 4.13.0 dan NumPy 2.4.6 di Python.

Empat teknik yang diuji: image blending (gabung dua gambar pake bobot), image negative (balik warna jadi negatif), transformasi logaritmik (stretch area gelap pake log), dan transformasi gamma (koreksi exposure pake power-law). Semua teknik dianalisis efeknya, termasuk perubahan histogram dan distribusi pixel.

Percobaan dilakukan di lingkungan Jupyter Notebook. Setiap teknik diimplementasi dalam sel kode terpisah, hasilnya divisualisasikan pake Matplotlib, dan dianalisis berdasarkan pengamatan visual serta perubahan nilai pixel. Fokusnya bukan cuma kodenya jalan, tapi lebih ke paham gimana operasi matematika ngaruh ke tampilan gambar.

Tujuan akhir dari tugas ini adalah bukan cuma ngerti cara nulis kode, tapi juga bisa milih teknik yang tepat buat masalah tertentu. Misal kalo ada gambar kegelapan, kita tau harus pake gamma atau log transform. Kalo mau gabung dua gambar, tau harus pake blending. Ini penting di dunia nyata soalnya manipulasi citra dipake di berbagai bidang: fotografi, medis, computer vision, desain grafis.

2 Citra yang Digunakan

Dua gambar dari `picsum.photos` dipake buat percobaan:

- **Citra 1:** gambar pemandangan, ukuran 600×800 pixel, 3 channel warna (BGR), tipe data `uint8`.
- **Citra 2:** gambar dengan konten berbeda, ukuran 768×1024 pixel, 3 channel (BGR), `uint8`.

Kedua gambar punya rentang intensitas penuh 0–255 (min 0, max 255). Citra 1 dominan hijau kebiruan (pemandangan alam), citra 2 warnanya lebih bervariasi. Ukurannya beda, jadi sebelum di-blending harus di-resize dulu.

3 Membaca Citra dan Konversi Warna

3.1 Membaca Citra

Citra dibaca pake `cv.imread()`. Hasilnya dicek:

- **Shape:** (tinggi, lebar, channel) – citra 1 punya (600, 800, 3), citra 2 punya (768, 1024, 3).

- **Dtype:** uint8, artinya tiap channel pake 8 bit dengan rentang 0–255.
- **Min/Max:** 0 sampe 255 buat kedua gambar, artinya distribusi pixel mencakup seluruh rentang.

3.2 Konversi BGR ke RGB

OpenCV baca gambar dalam format BGR (Blue-Green-Red). Ini beda sama Matplotlib yang pake RGB (Red-Green-Blue). Kalo gambar hasil `cv.imread()` langsung ditampilkan pake `plt.imshow()`, warnanya kacau: langit yang harusnya biru jadi merah, daun yang harusnya hijau jadi biru kehitaman. Ini terjadi karena channel merah (index 2) sama biru (index 0) tertukar.

Konversi dilakukan pake `cv.cvtColor(img, cv.COLOR_BGR2RGB)`. Fungsi ini mindahin channel biru ke index 2 dan channel merah ke index 0. Setelah konversi, tampilan warna jadi normal.

Pentingnya paham format warna: kalo kita mau manipulasi channel tertentu (misal nambahin merah), kita harus tau channel merah itu di index berapa. Di OpenCV channel merah itu index 2 (BGR), bukan index 0 (RGB). Kalo salah, hasil manipulasi ga sesuai ekspektasi.

4 Image Blending

4.1 Resize

Image blending butuh dua gambar ukuran sama soalnya operasi per-pixel. Citra 1 (600, 800) dan citra 2 (768, 1024) di-resize ke (600, 400) pake `cv.resize()`. Perhatikan: `cv.resize()` pake format (lebar, tinggi), jadi ukuran akhirnya (400, 600) dalam shape.

Resize ada konsekuensi: ngecilin ukuran (downscale) buang informasi, ngegedein (upscale) bikin gambar pecah. Apalagi kalo aspek ratio ga dipertahankan – proporsi objek berubah, lingkaran bisa jadi elips. Di sini aspek ratio diubah bebas soalnya prioritasnya ukuran sama, bukan proporsi.

4.2 Eksperimen Bobot

Rumus blending: $I_{\text{blend}} = \alpha I_1 + \beta I_2$ dengan $\alpha + \beta = 1$. Implementasi pake `cv.addWeighted()`. Tiga variasi bobot diuji:

α	β	Pengamatan
0.2	0.8	Citra 2 dominan banget. Warna dan tekstur sebagian besar dari citra 2. Citra 1 cuma keliatan samar di background kayak bayangan doang.
0.5	0.5	Dua gambar campur rata. Efeknya kayak overlay dengan transparansi 50%. Warna jadi perpaduan dua gambar. Detail dari kedua gambar masih keliatan.
0.8	0.2	Citra 1 dominan. Citra 2 keliatan tipis banget kayak lapisan transparan. Detail citra 2 hampir ga keliatan kalo ga diperhatiin.

Dari percobaan ini keliatan: α nentuin kontribusi citra 1, β buat citra 2. Makin gede α , makin dominan citra 1. Kalo $\alpha + \beta > 1$, intensitas bisa overflow melebihi 255 dan pixel jadi clipping. Kalo $\alpha + \beta < 1$, gambar jadi gelap karena intensitas rata-rata turun.

Di aplikasi nyata, blending dipake buat transisi video (crossfade antar scene), watermark transparan, efek superimpose, dan kompositing gambar.

5 Image Negative

Operasi negative: $I_{\text{negatif}} = 255 - I$. Ini operasi paling simpel. Pixel 0 (hitam) jadi 255 (putih), pixel 255 (putih) jadi 0 (hitam). Pixel 128 jadi 127.

Hasil yang diamati:

- Area terang di citra asli (langit, awan) jadi gelap di citra negatif.
- Area gelap di citra asli (pepohonan, bayangan) jadi terang.
- Histogram citra negatif adalah pencerminan horizontal dari histogram asli. Kalo di histogram asli ada puncak di intensitas 200 (terang), di histogram negatif puncaknya pindah ke intensitas 55 (200 jadi 55).

Image negative berguna buat analisis citra medis (sinar-X, MRI) karena detail tertentu lebih gampang dilihat di versi negatif. Juga dipake di fotografi film dan deteksi tepi.

Perlu dicatat: image negative itu cuma transformasi 1-to-1, artinya ga ada informasi yang hilang ato nambah. Pixel 200 pasti jadi 55, pixel 30 jadi 225. Ini beda sama log transform atau gamma yang distribusinya berubah secara non-linier. Negative itu cuma mindahin posisi pixel di rentang intensitas, bentuk histogramnya tetep sama cuma dicerminkan.

6 Transformasi Logaritmik

6.1 Implementasi

Rumus: $s = c \cdot \log(1 + r)$ dengan $c = 255 / \log(1 + r_{\text{max}})$. Dari percobaan, nilai konstanta $c = 105.89$. Setelah transformasi, rentang pixel tetap 0–255.

Ada temuan teknis pas implementasi: `np.max(img1_resize)` baliknya nilai bertipe `uint8`, jadi operasi $1 + 255$ overflow balik ke 0. Akibatnya $\log(1 + 0) = \log(1) = 0$ untuk denominator, c jadi tak terdefinisi. Di kode awal ini bikin hasil transformasi logaritmik error dan gambar keluar item semua. Solusinya pake `float(np.max(...))` biar operasi matematikanya bener.

6.2 Efek terhadap Citra

- **Area gelap:** kontras naik signifikan. Fungsi log punya slope curam di nilai rendah ($\frac{d}{dr} \log(1 + r) = \frac{1}{1+r}$, makin kecil r makin gede slope-nya). Perbedaan intensitas 5 jadi 10 di area gelap kelihatan jelas abis transformasi.
- **Area terang:** dikompresi. Slope log melandai di nilai besar, jadi perbedaan di highlight hampir ga kelihatan.

Distribusi pixel hasil transformasi: histogram lebih merata di rentang gelap, sementara di rentang terang rapat. Ini membuktikan bahwa transformasi logaritmik ngeregangin area gelap dan ngompres area terang.

6.3 Kapan Digunakan

Cocok buat gambar underexposed atau citra medis yang detailnya di area gelap. Tapi kalo gambar udah terang, malah ilangin detail. Jadi ga bisa dipake sembarangan.

7 Transformasi Gamma

7.1 Implementasi

Pixel dinormalisasi ke $[0, 1]$, dipangkat γ , balik ke $[0, 255]$. Empat nilai gamma diuji:

γ	Pengamatan
0.5	Gambar jadi lebih terang. Detail bayangan yang tadinya hampir item jadi keliatan. Kontras di area gelap meningkat. Histogram geser ke kanan (nilai intensitas lebih tinggi).
1.0	Sama persis kaya asli. Transformasi identitas, histogram ga berubah.
2.0	Gambar lebih gelap. Detail bayangan mulai ilang, kontras turun. Histogram geser ke kiri.
2.5	Gelap banget. Hanya area paling terang yang masih keliatan, sisanya item semua. Banyak detail ilang.

7.2 Analisis

- $\gamma < 1$: fungsi power-law punya slope > 1 di nilai rendah, jadi pixel gelap dinaikkan. Efeknya gambar terang, detail bayangan naik.
- $\gamma = 1$: $s = r$, identitas.
- $\gamma > 1$: fungsi power-law punya slope < 1 di nilai rendah, jadi pixel gelap makin gelap. Efeknya gambar gelap.

Transformasi gamma lebih fleksibel daripada log transform karena bisa dipake buat nge-terangin ($\gamma < 1$) dan nge-gelapin ($\gamma > 1$) gambar tinggal ganti nilai γ . Dipake di preprocessing computer vision, kalibrasi monitor, dan koreksi pencahayaan.

Kapan pake gamma dibanding log? Kalo kita cuma butuh narik detail gelap dan ga peduli area terang, log transform bisa jadi pilihan. Tapi kalo kita butuh kontrol penuh atas keseluruhan rentang intensitas, gamma lebih baik. Di computer vision, gamma correction sering dipake sebagai preprocessing step sebelum deteksi objek biar fitur lebih konsisten terlepas dari kondisi pencahayaan asli.

8 Analisis HOTS dan Studi Kasus

8.1 Perbandingan Teknik

Teknik	Operasi	Efek	Kapan Dipake
Blending	$\alpha I_1 + \beta I_2$	Gabung dua gambar pake bobot	Transisi video, watermark
Negative	$255 - I$	Balik intensitas pixel	Citra medis, fotografi
Log	$c \log(1 + r)$	Stretch area gelap, kompres terang	Underexposed, citra medis
Gamma	r^γ	Koreksi non-linier dua arah	Koreksi exposure, preprocessing

8.2 Pembahasan Hasil

Dari keempat teknik yang diuji, masing-masing nunjukkin karakteristik yang beda:

- **Image blending** menghasilkan gambar baru dari dua sumber. Ini satu-satunya teknik yang butuh dua input gambar. Efeknya linier – kalo α digeser 0.1, hasilnya juga berubah proporsional.
- **Image negative** ngebikin transformasi paling dramatis. Warna jadi kebalik total. Histogramnya juga kebalik sempurna. Sederhana tapi efeknya kuat.
- **Transformasi logaritmik** efeknya paling terasa di area gelap. Detail yang tadinya hampir ga keliatan jadi jelas. Tapi di area terang malah ilang detail. Jadi teknik ini juicynya spesifik.
- **Transformasi gamma** paling fleksibel. Bisa nerangin dan ngegelapin. Tapi efeknya tergantung distribusi pixel asli – dua gambar beda bisa ngasih hasil yang beda dengan nilai gamma yang sama.

Selain perbedaan teknis, keempat teknik ini juga punya pendekatan matematis yang beda. Blending dan negative adalah operasi linier – hasilnya proporsional sama input. Log transform dan gamma adalah non-linier – hubungan input-outputnya melengkung, jadi efeknya ga seragam di seluruh rentang intensitas. Ini penting dipahami soalnya kalo kita pengen efek yang seragam di semua pixel, pake operasi linier. Tapi kalo pengen target-in area tertentu (misal cuma area gelap), pake non-linier.

8.3 Analisis Pengambilan Keputusan

Citra terlalu gelap: Transformasi gamma dengan $\gamma < 1$ (misal 0.5) paling pas. Log transform juga bisa tapi efeknya kurang bisa diatur soalnya konstanta c tetap tergantung $r_{\max}$. Dengan gamma kita bisa atur nilai γ sesuai kebutuhan.

Citra terlalu terang: Transformasi gamma dengan $\gamma > 1$ (misal 2.0). Log transform ga cocok karena dia cuma stretch area gelap, bukan kompres area terang.

Menggabungkan dua citra: Image blending pake `cv.addWeighted()`. Atur α dan β sesuai proporsi yang diinginkan. Teknik lain (negative, log, gamma) ga relevan buat ini.

Detail area gelap: Log transform lebih spesifik karena slope fungsi log emang paling curam di nilai yang paling rendah. Tapi gamma < 1 juga oke dan hasilnya kadang lebih natural.

Apakah semua transformasi memperbaiki kualitas? Engga. Negative cuma mindahin posisi pixel, ga nambah informasi. Gamma > 2 bisa ngeblow-out detail. Transformasi itu alat, bukan solusi universal. Konteks dan tujuan yang nentuin.

8.4 Studi Kasus

Studi Kasus 1: Foto Malam Hari

Skenario: foto diambil malem hari, sebagian besar area gelap, detail hampir ga keliatan. Teknik yang tepat: transformasi gamma dengan $\gamma < 1$, misal 0.3–0.5.

Alasan: gamma < 1 naikin intensitas pixel rendah secara non-linier. Pixel yang tadinya di kisaran 10–30 (hampir item) bisa naik ke kisaran 50–120 (keliatan). Detail bayangan jadi lebih terlihat tanpa ngebuat area terang jadi washing out.

Risiko: noise sensor di area gelap ikut diperkuat. Noise yang tadinya ga keliatan (karena item) jadi kelihatan setelah gamma correction. Juga kalo gamma terlalu kecil (< 0.2), gambar bisa keliatan artificial dan pucat. Alternatifnya log transform, tapi gamma lebih fleksibel karena nilai γ bisa diatur.

Studi Kasus 2: Citra Medis

Skenario: citra medis (misal X-ray atau MRI) punya detail samar di area gelap yang penting buat diagnosis. Transformasi logaritmik bisa bantu.

Alasan: slope fungsi log paling curam di nilai rendah. Misal ada dua pixel bertetangga nilai 10 dan 15 di citra asli – beda tipis 5, hampir ga keliatan. Setelah log transform, nilai pixel jadi lebih beda soalnya slope di rentang itu gede. Detail yang tadinya samar jadi lebih kontras.

Risiko: sama kayak gamma, noise ikut diperkuat. Apalagi di citra medis noise bisa nyamarin detail yang sebenarnya penting. Juga transformasi ini cuma nerangin area gelap, ga ngubah area terang, jadi kalo citra medisnya udah punya rentang dinamis lebar, efeknya minimal.

Studi Kasus 3: Efek Gabungan Foto dan Tekstur

Skenario: desainer mau bikin efek overlay antara foto dan tekstur hujan. Teknik yang tepat: image blending.

Pengaruh bobot: α buat foto, β buat tekstur. $\alpha = 0.7, \beta = 0.3$ bikin tekstur keliatan tipis, efek hujan natural. $\alpha = 0.5, \beta = 0.5$ bikin tekstur dan foto sama kuat. α dan β harus ≤ 1 dan idealnya $\alpha + \beta = 1$ biar intensitas rata-rata tetap di rentang normal.

9 Kesimpulan dan Refleksi Pribadi

9.1 Kesimpulan

Semua teknik berhasil diimplementasi dan dianalisis:

- Image blending: kontrol gabung dua gambar lewat bobot α dan β .
- Image negative: inversi warna sederhana, histogram tercermin.
- Transformasi logaritmik: stretch efektif buat area gelap, tapu kompres area terang.
- Transformasi gamma: paling fleksibel, bisa koreksi dua arah.

Setiap teknik punya kelebihan dan kekurangan masing-masing. Ga ada yang universal terbaik – pemilihan tergantung kondisi gambar dan tujuan.

9.2 Refleksi Pribadi

Image blending dan negative paling gampang dipahami karena operasinya linier dan hasilnya bisa ditebak. Kalo α digedein, ya citra 1 makin dominan – simpel. Log transform dan gamma perlu eksperimen lebih karena efeknya tergantung distribusi pixel di gambar. Dua gambar beda bisa ngasih hasil yang beda dengan parameter yang sama.

Temuan teknis yang berharga: tipe data `uint8` gampang overflow kalo ga hati-hati. Pas ngitung konstanta log transform, $1 + \text{uint8}(255)$ overflow ke 0 karena NumPy maintain type `uint8` buat hasil operasinya. Akibatnya c jadi salah (negatif) dan gambar log item semua. Butuh waktu buat debug nemuin ini, tapi jadi pelajaran penting soal tipe data di NumPy.

Yang paling menarik dari tugas ini: pixel itu cuma angka 0–255, tapi cara kita mindahin dan ngubah angka-angka itu dampaknya gede banget ke tampilan gambar. Bentuk kurva fungsi – apakah linier, log, atau power-law – nentuin gimana distribusi pixel berubah. Ini kayak punya remote control buat brightness dan contrast, tapi lebih presisi dan bisa di-target ke rentang intensitas tertentu.

Kalo disuruh milih satu buat aplikasi peningkatan kualitas citra, saya bakal pake gamma correction karena paling fleksibel dan implementasinya sederhana. Tapi kalo butuh narik detail spesifik di area yang sangat gelap, bakal saya kombinasiin sama log transform.

Ponorogo, Juni 2026
Fatih Jawwad Al Muntaz